

Phase A.2 Gradebook Implementation Plan & Tracker

Implementation Strategy Options

Option A: Sequential Build (Recommended for Stability)

Duration: 3-4 weeks | **Risk:** Low | **Testing:** Incremental

- Week 1: Core models & database
- Week 2: Basic grade entry & calculations
- Week 3: Missing work escalation system
- Week 4: Advanced features & UI polish

Option B: Parallel Build (Faster but Higher Risk)

Duration: 2-3 weeks | **Risk:** Medium | **Testing:** Big bang

- All models built simultaneously
- Integration testing at the end
- Requires more coordination

Recommendation: Option A - Build incrementally on your solid Phase A foundation

Implementation Tracker

WEEK 1: Foundation & Core Models

Goal: Database schema and basic business logic

1.1 Database Models

☐ GradebookCategory Model

☐ `backend/app/models/gradebook_category.py`

☐ Fields: school_id, name, default_weight, grade_levels, is_required

☐ Methods: get_school_defaults(), apply_to_classroom()

☐ Assignment Model

☐ `backend/app/models/assignment.py`

☐ Fields: classroom_id, category_id, title, max_points, due_date, late_rules

☐ Optional: lesson_plan_id, standard_id for Phase B prep

☐ Methods: calculate_late_penalty(), is_overdue()

☐ Grade Model

☐ `backend/app/models/grade.py`

☐ Fields: assignment_id, student_id, points_earned, status, submitted_date

☐ Status enum: GRADED, MISSING, LATE, EXCUSED, IN_PROGRESS

☐ Audit fields: graded_by, graded_at, change_reason, parent_visible

☐ **ClassroomGradebook Model**

☐ `backend/app/models/classroom_gradebook.py`

☐ Fields: `classroom_id`, `categories_config` (JSON), `release_schedule`, `late_policies`

☐ Methods: `calculate_student_grade()`, `get_missing_work()`

☐ **GradeScale Model**

☐ `backend/app/models/grade_scale.py`

☐ Fields: `school_id`, `grade_level`, `scale_type`, `scale_definition` (JSON)

☐ Support for: Number scales (K-3), Letter grades (4+), Custom scales

1.2 Migration & Seeding

☐ **Create Migration File**

☐ `backend/alembic/versions/phase_a2_gradebook.py`

☐ All gradebook tables with proper indexes

☐ Foreign key constraints to Phase A tables

☐ **Seed Default Data**

☐ Default categories: Tests (50%), Quiz-Classwork (40%), Homework (10%)

☐ Default grade scales: K-3 (1-4), 4+ (A-F)

☐ Default escalation thresholds

1.3 Basic Schemas

☐ **Pydantic Schemas**

☐ `backend/app/schemas/gradebook.py`

☐ Create, Update, and Output schemas for all models

☐ Validation rules for weights, grades, dates

Week 1 Exit Criteria:

- ☒ Database schema created and migrated
 - ☒ Models pass basic CRUD tests
 - ☒ Default data seeded successfully
-

WEEK 2: Grade Entry & Calculations

Goal: Core gradebook functionality working

2.1 Grade Calculation Engine

☐ **Calculation Service**

☐ `backend/app/services/grade_calculator.py`

☐ `calculate_category_grade()` - weighted averages with drop-lowest

☐ `calculate_overall_grade()` - category weights applied

☐ `handle_missing_work()` - include as zero vs. exclude logic

☐ `apply_late_penalties()` - per assignment late rules

- ☐ **Real-time Updates**
- ☐ Grade change triggers immediate recalculation
- ☐ Background task for bulk updates
- ☐ Cache frequently accessed calculations

2.2 Assignment Management APIs

- ☐ **Assignment Router**
- ☐ `backend/app/routers/assignments.py`
- ☐ CRUD operations for assignments
- ☐ Bulk create from templates
- ☐ Link to lesson plans (Phase B prep)
- ☐ **Category Management APIs**
- ☐ `backend/app/routers/gradebook_categories.py`
- ☐ School default management
- ☐ Teacher classroom overrides
- ☐ Mid-year category changes handling

2.3 Grade Entry APIs

- ☐ **Grade Router**
- ☐ `backend/app/routers/grades.py`
- ☐ Individual grade entry with validation
- ☐ Bulk grade entry (CSV/JSON import)
- ☐ Grade change audit logging
- ☐ Parent visibility controls

2.4 Basic Frontend Interface

- ☐ **Teacher Gradebook Page**
- ☐ `frontend/src/pages/TeacherGradebook.jsx`
- ☐ Spreadsheet-like grade entry grid
- ☐ Category weight configuration
- ☐ Assignment creation modal
- ☐ **Admin Gradebook Config**
- ☐ Add gradebook section to Admin.jsx
- ☐ Category defaults management
- ☐ Grade scale configuration
- ☐ Release schedule settings

Week 2 Exit Criteria:

- ☒ Teachers can create assignments and enter grades
- ☒ Grade calculations work correctly with category weights
- ☒ Admin can configure school defaults

- ☒ Basic gradebook interface functional
-

WEEK 3: Missing Work Escalation System

Goal: Intelligent automated notifications working

3.1 Missing Work Models

☐ **MissingWorkLog Model**

☐ `backend/app/models/missing_work_log.py`

☐ Fields: assignment_id, student_id, days_missing, escalation_level

☐ Track notification history and grace periods

☐ **EscalationConfig Model**

☐ `backend/app/models/escalation_config.py`

☐ Admin-configurable thresholds per grade level

☐ Notification method preferences

☐ School break calendars

☐ **StudentEscalationOverride Model**

☐ `backend/app/models/student_escalation_override.py`

☐ Teacher/admin overrides per student

☐ Special needs modifications

☐ Temporary escalation pauses

3.2 Escalation Engine

☐ **Escalation Service**

☐ `backend/app/services/escalation_service.py`

☐ Daily batch job to check missing work

☐ Grace period calculations (absence days)

☐ Smart batching (daily digests vs. immediate alerts)

☐ School break detection

☐ **Notification Service**

☐ `backend/app/services/notification_service.py`

☐ Email templates for different escalation levels

☐ SMS integration (future)

☐ Parent vs. student notification routing

3.3 Missing Work APIs

☐ **Missing Work Router**

☐ `backend/app/routers/missing_work.py`

☐ Get missing work by student/teacher/admin

☐ Manual escalation triggers

☐ Override management

☐ **Escalation Config Router**

- ☐ backend/app/routers/escalation_config.py
- ☐ Admin configuration interface
- ☐ Threshold management
- ☐ Notification preferences

3.4 Missing Work Frontend

- ☐ **Missing Work Dashboard**
- ☐ frontend/src/components/MissingWorkDashboard.jsx
- ☐ Teacher view: missing work by class
- ☐ Student view: personal missing work list
- ☐ Parent view: child's missing work summary
- ☐ **Admin Escalation Config**
- ☐ Add to Admin.jsx
- ☐ Threshold configuration interface
- ☐ Notification template management
- ☐ Override management tools

Week 3 Exit Criteria:

- ☒ Missing work automatically detected and logged
 - ☒ Escalation notifications sent based on configured thresholds
 - ☒ Grace periods calculated correctly for absences
 - ☒ Admin can configure escalation rules
 - ☒ Teachers can override escalation for individual students
-

WEEK 4: Advanced Features & Polish

Goal: Production-ready gradebook system

4.1 Advanced Grade Features

- ☐ **Extra Credit System**
- ☐ Extra credit assignments and calculations
- ☐ Bonus points on regular assignments
- ☐ Category-level extra credit handling
- ☐ **Grade Import/Export**
- ☐ CSV import for bulk grade entry
- ☐ Excel template downloads
- ☐ Grade export for report cards
- ☐ **Assignment Templates**
- ☐ Save assignment as template
- ☐ School-wide template library
- ☐ Quick assignment creation from templates

4.2 Reporting & Analytics

☐ Grade Reports

- ☐ `backend/app/services/grade_reports.py`
- ☐ Student progress reports
- ☐ Class performance summaries
- ☐ Missing work reports
- ☐ Grade change audit reports

☐ Parent/Student Portal

- ☐ `frontend/src/pages/ParentPortal.jsx`
- ☐ Grade visibility with real-time/scheduled release
- ☐ Missing work notifications
- ☐ Progress trend charts

4.3 Integration & Performance

☐ Phase B Preparation

- ☐ Assignment-lesson plan linking ready
- ☐ Standards tracking foundation
- ☐ Curriculum mapping preparation

☐ Performance Optimization

- ☐ Database query optimization
- ☐ Grade calculation caching
- ☐ Bulk operation improvements

4.4 Testing & Documentation

☐ Comprehensive Testing

- ☐ Unit tests for all calculation logic
- ☐ Integration tests for escalation system
- ☐ Performance tests with realistic data

☐ User Documentation

- ☐ Teacher gradebook guide
- ☐ Admin configuration manual
- ☐ Parent portal instructions

Week 4 Exit Criteria:

- ☒ All Phase A.2 features complete and tested
 - ☒ Performance benchmarks met
 - ☒ Integration with Phase A working seamlessly
 - ☒ Ready for teacher adoption and feedback
 - ☒ Foundation prepared for Phase B integration
-

Success Metrics & Validation

Performance Benchmarks

- ☐ **Grade Entry Speed:** 25 students, 1 assignment $\leq$ 2 minutes
- ☐ **Calculation Speed:** Overall grade recalculation $\leq$ 1 second for 30 students
- ☐ **Missing Work Detection:** Daily batch job completes $\leq$ 5 minutes for 1000 students
- ☐ **Notification Delivery:** Escalation emails sent $\leq$ 30 seconds after trigger

Functional Validation

- ☐ **Category Weights:** Calculations accurate to 2 decimal places
- ☐ **Late Penalties:** Applied correctly per teacher-defined rules
- ☐ **Grace Periods:** Absence days properly extend due dates
- ☐ **Escalation Logic:** Notifications sent at correct thresholds
- ☐ **Admin Overrides:** All configuration changes take effect immediately

User Acceptance Criteria

- ☐ **Teachers:** Can set up gradebook and enter grades efficiently
 - ☐ **Admin:** Can configure all settings without developer help
 - ☐ **Parents:** Receive timely, actionable missing work notifications
 - ☐ **Students:** Can see current grades and missing work clearly
-

Integration Points with Phase A

Existing Models Used

- ☒ **Classroom** → gradebooks belong to classrooms
- ☒ **Enrollment** → students auto-added to gradebooks
- ☒ **ClassroomTeacherAssignment** → permission controls for grade entry
- ☒ **AcademicYear** → gradebooks scoped to academic years
- ☒ **Student** → grade recipients
- ☒ **ParentStudentRelationship** → notification routing

New Relationships Added

- **Assignment** → belongs_to classroom, has_many grades
 - **Grade** → belongs_to assignment and student
 - **ClassroomGradebook** → belongs_to classroom, has_many assignments
 - **MissingWorkLog** → tracks missing work per student/assignment
-

Post-Phase A.2 Roadmap

Phase A.3: Attendance System (Week 5-6)

- Integrate with gradebook for attendance-grade correlation
- Use classroom structure for daily attendance taking
- Link to missing work (absent = grace period)

Phase A.4: Scheduling System (Week 7-8)

- Time-based class periods
- Integration with gradebook (attendance per period)
- Specialist pull-out scheduling

Phase A.5: Integration & Analytics (Week 9-10)

- Cross-system reporting and analytics
- Teacher performance comparisons
- Student progression tracking

? Potential Risks & Mitigation

Technical Risks

- **Grade calculation complexity** → Start with simple weighted averages, add features incrementally
- **Real-time performance** → Implement caching and background processing
- **Data migration** → Extensive testing with realistic data volumes

User Adoption Risks

- **Teacher resistance to change** → Gradual rollout with training
- **Parent notification overload** → Smart batching and configurable thresholds
- **Admin configuration complexity** → Simple UI with good defaults

Integration Risks

- **Phase A compatibility** → Maintain all existing functionality
- **Phase B preparation** → Design APIs for lesson plan integration
- **Performance impact** → Monitor and optimize database queries

Mitigation Strategy: Incremental delivery with testing at each milestone, maintaining backward compatibility, and getting user feedback early and often.

✅ Ready to Begin Implementation

Missing anything? This plan covers:

- ☒ Complete gradebook functionality
- ☒ Intelligent missing work escalation
- ☒ Admin configurability
- ☒ Integration with Phase A foundation
- ☒ Performance and testing requirements
- ☒ Clear weekly milestones
- ☒ Success criteria and validation

Choose your implementation approach and let's build the future of gradebook systems! 🚀